

Material de Estudo Completo: Estruturas de Dados e Linguagem C

Guia Definitivo Teórico e Prático para Provas e Concursos

Tópicos: Pilhas, Filas, Ponteiros, Gerenciamento de Memória & Manipulação de Arquivos

Baseado em: LE02 e LE03 (Engenharia / Ciência da Computação)

Este guia compila com rigor técnico, didática aprofundada e riqueza de detalhes toda a matéria necessária para o domínio completo das listas de exercícios e provas sobre **Ponteiros**, **Alocação Dinâmica**, **Arquivos (Streams e Binários)**, **Pilhas (LIFO)** e **Filas (FIFO)**. Estude este material para sanar desde as dúvidas mais elementares até sutilezas avançadas de arquitetura de memória em C.

Módulo 1: Pilhas (Stacks)

A pilha é uma estrutura de dados linear abstrata regida pelo princípio **LIFO (Last-In, First-Out)**: o último elemento inserido é impreterivelmente o primeiro a ser removido. Todas as operações de mutação ocorrem em uma única extremidade denominada **topo**.

1.1 Operações Fundamentais

- `push(dado)` : Adiciona um elemento ao topo da pilha. Complexidade de tempo: **O(1)**.
- `pop()` : Remove e retorna o elemento situado no topo. Se a pilha estiver vazia, gera condição de *Underflow*. Complexidade: **O(1)**.
- `peek()` ou `top()` : Inspecciona e retorna o valor do topo *sem desempilhá-lo* ou alterar o ponteiro de controle. Complexidade: **O(1)**.
- `isEmpty()` : Verifica se a pilha está desprovida de elementos (`topo == -1` ou `NULL`).
- `isFull()` : Exclusivo para pilhas estáticas; avalia se o array atingiu a capacidade máxima cadastrada.

1.2 Pilhas Estáticas vs. Pilhas Dinâmicas

Característica	Pilha Estática (Vetor / Array)	Pilha Dinâmica (Nós Encadeados)
Alocação de Memória	Compilação / Stack da memória com tamanho pré-fixado (<code>MAX</code>).	Execução (Heap) via <code>malloc()</code> para cada novo nó.
Controle de Topo	Inteiro inicializado em <code>-1</code> . Indica índice vazio e permite pré-incremento <code>++topo</code> .	Ponteiro para nó inicializado com <code>NULL</code> .
Limitações	Susceptível a <i>Stack Overflow</i> se ultrapassar a capacidade física.	Gasta memória adicional com ponteiros (<code>sizeof(struct No*)</code>) e exige <code>free()</code> .

Por que 'topo' inicia em -1 na implementação estática?

Em C, vetores são indexados a partir do zero (0). Se inicializássemos `topo = 0`, estaríamos afirmando falsamente que o elemento no índice 0 é o topo válido. Inicializando em `-1`, a pilha vazia é modelada com precisão matemática, permitindo que a inserção use `vetor[++topo] = v;` (o índice é incrementado para 0 antes da gravação).

1.3 Rastreamento Passo a Passo (Exercício Típico)

Considere uma pilha inicialmente vazia submetida à seguinte sequência: `push(10)`, `push(20)`, `pop()`, `push(30)`, `push(40)`, `pop()`, `push(50)`.

- `push(10)` → Pilha: [10] (Topo: 10)
- `push(20)` → Pilha: [10, 20] (Topo: 20)
- `pop()` → Remove 20. Pilha: [10] (Topo: 10)
- `push(30)` → Pilha: [10, 30] (Topo: 30)
- `push(40)` → Pilha: [10, 30, 40] (Topo: 40)
- `pop()` → Remove 40. Pilha: [10, 30] (Topo: 30)
- `push(50)` → Pilha: [10, 30, 50] (Topo: 50)

Resultado final: O elemento no topo é **50** e a pilha contém exatamente **3 elementos** remanescentes.

1.4 Aplicações Clássicas de Pilhas

- **Validação de Parênteses e Escopo:** Empilha caracteres de abertura (`(`, `[`, `{`) e, ao encontrar um fechamento, desempilha e compara. Se tentar desempilhar com pilha vazia ou terminar a leitura com itens na pilha, a expressão é inválida (ex: `"((a+b)"` → inválida (0); `"((a+b)*c)"` → válida (1)).
- **Mecanismo Desfazer / Refazer (Undo/Redo):** Salva estados prévios em pilhas independentes.
- **Histórico de Navegação:** O botão "Voltar" do browser desempilha a URL visitada imediatamente anterior.
- **Call Stack (Pilha de Chamadas do Sistema):** Guarda o endereço de retorno, variáveis locais e parâmetros em chamadas recursivas e funções.
- **Algoritmos de Busca em Profundidade (DFS / Backtracking):** Resolução de labirintos, xadrez e jogos de busca recursiva.

Módulo 2: Filas (Queues)

A fila opera sob o princípio **FIFO (First-In, First-Out)**: o primeiro elemento a entrar é o primeiro a ser processado e retirado. Possui duas extremidades ativas: **frente (head/início)** para remoção e **fim (tail/cauda)** para inserção.

2.1 Operações Fundamentais

- `enqueue(dado)` : Enfileira um novo elemento ao final da fila. Complexidade: **O(1)**.
- `dequeue()` : Desenfileira e remove o elemento situado na frente da fila. Complexidade: **O(1)**.
- `peek()` / `front()` : Retorna o dado na frente da fila sem removê-lo.

2.2 Fila Linear Simples vs. Buffer Circular

O Problema da Fila Linear Simples:

Em um vetor tradicional, se apenas incrementarmos o índice `fim` na inserção e o índice `inicio` na remoção, as posições iniciais liberadas se tornam *inutilizáveis*. Quando `fim` atinge `MAX - 1`, a fila acusa estar "cheia", mesmo que as posições 0, 1 etc. estejam completamente desocupadas!

Solução Otima: Fila Circular (Circular Buffer)

Aplica-se o operador de resto da divisão (módulo `%`) para que os ponteiros girem e voltem ao índice 0 ao ultrapassar o fim do array:

```
fim = (fim + 1) % MAX; | inicio = (inicio + 1) % MAX;
```

2.3 Filas Dinâmicas Encadeadas

Para implementar uma fila dinâmica com complexidade **O(1)** tanto na inserção quanto na remoção, a estrutura de dados principal precisa gerenciar impreterivelmente **dois ponteiros externos**:

- `inicio (head)` : Aponta para o primeiro elemento (usado no `dequeue`).
- `fim (tail)` : Aponta para o último nó inserido (usado no `enqueue` para evitar ter que percorrer a lista inteira).

2.4 Comparativo Estrutural Definitivo: Pilha vs. Fila

Aspecto de Projeto	Pilha (Stack)	Fila (Queue)
Princípio Fundamental	LIFO (Last-In, First-Out)	FIFO (First-In, First-Out)
Pontos de Acesso	Apenas uma extremidade ativa (o Topo).	Duas extremidades ativas (Frente/Início e Cauda/Fim).
Controle de Ponteiros Dinâmicos	1 único ponteiro externo (<code>topo</code>).	2 ponteiros de controle (<code>inicio</code> e <code>fim</code>).
Aplicações Típicas	Desfazer (Undo), Parsers, Recursão (Call Stack).	Spooler de impressão, filas de mensagens, Round-Robin.

Módulo 3: Ponteiros, Aritmética e Alocação Dinâmica

Ponteiros são variáveis cujo conteúdo é o endereço físico de memória RAM de outra variável. Dominar sua sintaxe e ciclo de vida é pré-requisito indispensável em C.

3.1 Operadores de Ponteiros

- **Operador de Endereço (`&`)**: Retorna o endereço hexadecimal onde a variável reside na memória.
- **Operador de Desreferenciação / Conteúdo (`*`)**: Permite acessar ou modificar o valor armazenado na posição apontada. Exemplo: `*ptr = 50;` modifica a variável referenciada por `ptr` .

Atenção à Sintaxe de Atribuição:

`int *ptr; ptr = №` → **CORRETO** : O ponteiro recebe o endereço de `numero` .

`*ptr = №` → **ERRO FATAL** : Tenta gravar um endereço de memória dentro do valor apontado.

`ptr = 100;` → **ERRO FATAL** : Faz o ponteiro apontar para o endereço literal 100 na RAM, gerando falha de segmentação ao desreferenciar.

3.2 Aritmética de Ponteiros

A aritmética de ponteiros em C calcula deslocamentos baseados no tamanho do tipo de dado subjacente (`sizeof(tipo)`):

```
int vetor[4] = {10, 20, 30, 40};
int *p = vetor; // p aponta para vetor[0] (conteúdo: 10)
p = p + 2;      // Desloca o ponteiro em 2 * sizeof(int) bytes
printf("%d", *p); // Imprime: 30 (que reside no índice 2)
```

3.3 Alocação Dinâmica no Heap: malloc vs. calloc vs. free

Função	Assinatura & Comportamento	Diferença Prática Crucial
<code>malloc()</code>	<code>void* malloc(size_t bytes);</code>	Aloca um bloco contíguo de bytes no Heap. O conteúdo permanece indefinido (lixo de memória).
<code>calloc()</code>	<code>void* calloc(size_t n, size_t size);</code>	Aloca memória para <i>n</i> itens de tamanho <i>size</i> e inicializa todos os bits com ZERO absoluto.
<code>free()</code>	<code>void free(void* ptr);</code>	Devolve o bloco ao Heap. Deve ser chamado sempre para prevenir Vazamentos de Memória (Memory Leaks).

Boas Práticas de Ouro no Uso de Memória Dinâmica:

- 1. **Sempre testar se a alocação retornou NULL:** Se o sistema não tiver memória livre, `malloc` / `calloc` retorna `NULL`.
- 2. **Evitar Ponteiros Órfãos/Selvagens (Dangling Pointers):** Imediatamente após executar `free(ptr);`, atribua `ptr = NULL;`. Isso impede acessos indevidos posteriores.
- 3. **Nunca retornar ponteiros para vetores estáticos locais:** Variáveis declaradas na Stack dentro de uma função deixam de existir quando a função retorna.

Módulo 4: Manipulação de Arquivos (Streams & I/O)

A manipulação de arquivos em C é feita através do tipo opaco `FILE` definido em `<stdio.h>`, operando por meio de buffers intermediários no sistema operacional.

4.1 Modos de Abertura com fopen()

Modo	Ação se o Arquivo Já Existe	Ação se o Arquivo Não Existe	Operação
<code>"r"</code>	Abre para leitura a partir do início.	Falha e retorna NULL!	Leitura
<code>"w"</code>	Trunca e apaga todo o conteúdo prévio!	Cria um novo arquivo em branco.	Escrita
<code>"a"</code>	Preserva os dados; posiciona ponteiro no fim.	Cria um novo arquivo em branco.	Anexo (Append)
<code>"rb" / "wb"</code>	Modos binários (não realizam conversão de caracteres <code>\n</code> / <code>\r</code>).	Binário puro	

4.2 Funções Essenciais de Leitura e Escrita

- **Arquivos Texto:** `fprintf()`, `fscanf()`, `fputs()`, `fgets()`, `fputc()`, `fgetc()`.
- **Arquivos Binários:** `fwrite(&dados, sizeof(Tipo), qtd, arq)` e `fread(&dados, sizeof(Tipo), qtd, arq)` — gravam e leem diretamente blocos de structs da RAM sem formatação texto.
- **A Macro `EOF`:** Definida em `<stdio.h>`, representa uma constante inteira negativa (tipicamente `-1`) indicando que o fluxo de leitura atingiu o final do arquivo ou um erro de I/O.

4.3 Posicionamento e Fechamento Seguro

- `fseek(FILE *fp, long offset, int whence)`: Reposiciona o cursor no arquivo. Parâmetros de `whence`: `SEEK_SET` (início), `SEEK_CUR` (posição corrente), `SEEK_END` (final do arquivo).
- `ftell(FILE *fp)`: Retorna o deslocamento numérico atual do ponteiro em bytes desde o início do arquivo. Muito utilizado para determinar o tamanho total de arquivos (`fseek(fp, 0, SEEK_END); long tam = ftell(fp);`).
- `fclose(FILE *fp)`: **Crítico e Obrigatório:** Descarrega (flush) todos os buffers de memória pendentes para a mídia física de armazenamento e devolve os descritores de arquivo (File Descriptors) ao Sistema Operacional.

Módulo 5: Análise de Códigos com Bugs e Soluções Oficiais

Abaixo estão as correções detalhadas dos 8 cenários de laboratório apresentados nas avaliações práticas.

Cenário 1: Desempilhar com Underflow (LE03 - Listagem 1.1)

CÓDIGO ERRADO

```
int pop() {
    // ERRO: Não valida se pilha está vazia!
    // Se topo == -1, acessará pilha[-1]!
    return pilha[topo--];
}
```

CORREÇÃO ROBUSTA

```
int pop() {
    if (topo == -1) {
        printf("Erro: Stack Underflow!");
        return -1;
    }
    return pilha[topo--];
}
```

Explicação Técnica: O código original decrementava o índice sem checar se já estava na base, acessando posições de memória não pertencentes ao vetor (Undefined Behavior).

Cenário 2: Desempilhamento Dinâmico com Vazamento de Memória (LE03 - Listagem 2.1)

CÓDIGO ERRADO

```
int pop() {
    if (topo == NULL) return -1;
    int valor = topo->dado;
    topo = topo->proximo; // ERRO GRAVE!
    // O nó foi desconectado sem free()!
    return valor;
}
```

CORREÇÃO ROBUSTA

```
int pop() {
    if (topo == NULL) return -1;
    No* temp = topo;
    int valor = temp->dado;
    topo = topo->proximo;
    free(temp); // Desaloca a memória RAM
    return valor;
}
```

Explicação Técnica: Salva-se o endereço do nó a ser descartado em uma variável temporária `temp`, atualiza-se o ponteiro `topo` e chama-se obrigatoriamente `free(temp)`.

Cenário 3: Fila Estática com Estouro por Aritmética Linear (LE03 - Listagem 3.1)

CÓDIGO ERRADO

```
void enqueue(int v) {
    if (total >= MAX) return;
    fila[++fim] = v; // ERRO: estoura vetor
    total++;
}

int dequeue() {
    if (total == 0) return -1;
    total--;
    return fila[inicio++]; // Desperdiça slots
}
```

CORREÇÃO ROBUSTA

```
void enqueue(int v) {
    if (total >= MAX) return;
    fim = (fim + 1) % MAX; // Circular
    fila[fim] = v;
    total++;
}

int dequeue() {
    if (total == 0) return -1;
    int v = fila[inicio];
    inicio = (inicio + 1) % MAX; // Circular
    total--;
    return v;
}
```

Explicação Técnica: O uso do operador módulo (`% MAX`) permite que os índices voltem a reaproveitar o espaço do início do array de forma segura.

Cenário 4: Desenfileiramento com Ponteiro Órfão e Vazamento (LE03 - Listagem 4.1)

CÓDIGO ERRADO

```
void dequeue_errado() {
    if (inicio == NULL) return;
    if (inicio == fim) {
        inicio = NULL; // ERRO: fim fica órfão
    } else {
        inicio = inicio->prox;
    }
    // Falta free() no nó removido!
}
```

CORREÇÃO ROBUSTA

```
int dequeue() {
    if (inicio == NULL) return -1;
    No* temp = inicio;
    int v = temp->dado;
    inicio = inicio->prox;
    if (inicio == NULL) fim = NULL; // Ajusta fim
    free(temp); // Libera o nó da memória
    return v;
}
```

Explicação Técnica: Quando a fila esvazia após a remoção do último elemento, tanto `inicio` quanto `fim` devem apontar para `NULL`, e o bloco de memória precisa ser desalocado com `free()`.

Cenário 5: Criação e Escrita com Modo Incorreto e Falta de Validação (LE02 - Listagem 1.1)

CÓDIGO ERRADO

```
// Modo "r" abre para leitura, não escrita!  
FILE *arq = fopen("notas.txt", "r");  
float nota1 = 8.5, nota2 = 9.2;  
// Escreve sem validar se arq != NULL  
fprintf(arq, "Nota 1: %.2f", nota1);  
// Faltou fclose()  
return 0;
```

CORREÇÃO ROBUSTA

```
FILE *arq = fopen("notas.txt", "w"); // Modo "w"  
if (arq == NULL) {  
    perror("Erro ao abrir arquivo");  
    return 1;  
}  
float nota1 = 8.5, nota2 = 9.2;  
fprintf(arq, "Nota 1: %.2f", nota1);  
fprintf(arq, "Nota 2: %.2f", nota2);  
fclose(arq); // Sempre fechar!  
return 0;
```

Explicação Técnica: Para criar e escrever dados, usa-se o modo `"w"` (ou `"a"`). Se a chamada a `fopen` falhar, tentar escrever em um ponteiro nulo gerará falha de segmentação.

Cenário 6: Atribuição Errada de Ponteiros e Escrita Indireta (LE02 - Listagem 3.1)

CÓDIGO ERRADO

```
int numero = 50;  
int *ptr;  
*ptr = 100; // ERRO: *ptr não foi alocado!  
ptr = 100; // ERRO: sobrescreve endereço!  
printf("Valor: %d", ptr);
```

CORREÇÃO ROBUSTA

```
int numero = 50;  
int *ptr;  
ptr = &numero; // ptr recebe endereço de numero  
*ptr = 100; // Modifica o conteúdo apontado  
printf("Valor: %d", *ptr); // Imprime 100
```

Explicação Técnica: Para associar o ponteiro ao endereço, usa-se `ptr = &numero`. Para alterar o valor referenciado, usa-se o operador de desreferenciação `*ptr = 100;`.

Cenário 7: Retorno de Vetor Local da Pilha de Execução / Stack (LE02 - Listagem 4.1)

CÓDIGO ERRADO

```
int* gerarVetor() {
    int dados[3] = {10, 20, 30};
    return dados; // ERRO GRAVE! Memória destruída
}

int main() {
    int *meuVetor = gerarVetor();
    printf("%d", meuVetor[0]); // Lixo de memória!
}
```

CORREÇÃO ROBUSTA (HEAP)

```
int* gerarVetor() {
    int *dados = (int*) malloc(3 * sizeof(int));
    if (dados == NULL) return NULL;
    dados[0] = 10; dados[1] = 20; dados[2] = 30;
    return dados; // Seguro: memória Heap persiste
}

int main() {
    int *meuVetor = gerarVetor();
    if (meuVetor != NULL) {
        printf("%d", meuVetor[0]);
        free(meuVetor); // Libera no main
    }
}
```

Explicação Técnica: Vetores locais são alocados na *Stack Frame* da função e destruídos no término da chamada. Para retornar um vetor de uma função, ele precisa ser alocado dinamicamente no *Heap* com `malloc()`.

Módulo 6: Checklist para a Prova (Resumo Relâmpago)

- **Pilhas:** LIFO. Inserção e remoção no mesmo ponto (`topo`). Topo inicial em array: `-1`.
- **Filas:** FIFO. Inserção no `fim`, remoção no `inicio`. Fila estática linear sofre perda de posições; resolvida com buffer circular (`% MAX`). Fila dinâmica necessita de 2 ponteiros de controle.
- **Ponteiros:** `&` extrai endereço; `*` desreferencia/acessa conteúdo. `ptr + 1` avança o número de bytes do tipo da variável.
- **Memória:** `malloc` aloca lixo; `calloc` aloca zerado; `free` evita memory leak. Após `free(p)`, faça `p = NULL` para não deixar ponteiro pendente.
- **Arquivos:** `"r"` requer arquivo existente; `"w"` sobrescreve/trunca; `"a"` preserva e concatena no fim. Sempre validar se `fopen() != NULL` e nunca esquecer de fechar com `fclose()`. Para structs em binário, use `fread` e `fwrite`.